

IDENTITY + ACCESS

Who are *you?*

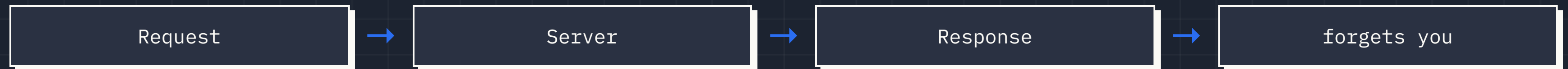
HTTP forgets you the instant it answers. Authentication is every trick we use to be remembered — and to prove it’s really you.

- cookies
- sessions
- JWT
- OAuth
- Auth0

STATELESSNESS

HTTP has no memory.

Every request stands alone. The server answers, then forgets. A logged-in user and a total stranger send requests that look identical.



so how does the next request know it's still you?

THE VOCABULARY

Authentication is not authorization.

AUTHENTICATION – AUTHN

Who are you?

Proving identity. Are you logged in, and as whom? A login form answers this.

AUTHORIZATION – AUTHZ

What may you do?

Checking permission. You're logged in — but are you allowed to delete this quote?

most breaches aren't broken logins — they're missing permission checks.

COOKIES

A cookie is a note the browser keeps.

```
// server sets it once
res.cookie("user", "ada")

// browser sends it back on
// EVERY later request — automatically
GET /quotes
Cookie: user=ada
```

THE MECHANISM

- ✓ Server replies with Set-Cookie.
- ✓ Browser stores it for that site.
- ✓ Browser re-attaches it every request.
- ✓ No extra code — it's automatic.

now the server can recognize a returning browser.

THE COOKIE PROBLEM

But a cookie is just editable text.

WHAT THE USER STORES

```
Cookie: user=ada

// ...so the user edits it to:
Cookie: user=admin
```

WHY THAT'S BAD

- ✗ Anyone can read a plain cookie.
- ✗ Anyone can forge one.
- ✗ The server has no way to tell.
- ✗ Identity you can't trust is useless.

we can recognize the browser — but we can't yet trust what it claims.

SESSIONS

Keep the data on the server. Not the cookie.

```
// server holds the real data
sessions["a1b2c3"] = { userId: 7 }

// cookie holds only a random id
Set-Cookie: sid=a1b2c3

// later: look the id up
req.session = sessions[sid]
```

WHY THIS IS SAFE

- ✓ The id is random and meaningless.
- ✓ Forging it points to nothing.
- ✓ Real data never leaves the server.
- ✓ Change sid and you're simply logged out.

the cookie is a claim check. the server holds the coat.

THE SESSION COST

Now the server has to remember everyone.

THE PRICE OF SESSIONS

- ✗ Store every active session.
- ✗ Look one up on every request.
- ✗ Share the store across servers.
- ✗ More users, more memory.

A NEW QUESTION

Could the server prove it's you without storing anything at all — trusting the request itself?

that question is exactly what a JWT answers.

JWT — JSON WEB TOKEN

A token that carries its own proof.

Instead of an id pointing at server data, the token is the data — plus a signature that proves it hasn't been touched.

Header

Payload

Signature

three base64 parts, joined by dots. the signature is the whole trick.

READABLE, BUT TAMPER-PROOF

Anyone can read it. Only the issuer can sign it.

```
// the decoded payload – plain to read
{
  "sub": "auth0|64a1bc23...", // user id
  "email": "ada@example.com",
  "iat": 1717000000,           // issued
  "exp": 1717086400           // expires
}
```

READ THE FINE PRINT

- ✓ Payload is encoded, not encrypted.
- ✓ The signature can't be faked without the secret key.
- ✓ The server verifies it with a public key.

So never put a password or secret in the payload.

TWO WAYS TO BE REMEMBERED

Store it, or sign it.

Question	Session	JWT
Where's the data?	On the server	Inside the token
Every request	Look it up in the store	Verify the signature
To log out	Delete the session	Let the token expire

same job — prove who you are. they just trade storage for verification.

WHY NOT BUILD IT YOURSELF?

Real auth is a security minefield.

YOU'D HAVE TO GET ALL OF THIS RIGHT

- ✗ Password hashing + salting
- ✗ Brute-force protection
- ✗ Password reset flows
- ✗ Email verification
- ✗ 2FA / MFA

AND THE STAKES

Get any one wrong and you leak real people's accounts. This is why almost no company hand-rolls its own login anymore.

the mechanism was the easy part. the safety is the hard part.

IDENTITY AS A SERVICE

Let a specialist run the login.

Auth0 hosts the login page, stores and hashes credentials, and hands your app a signed JWT. Your job shrinks to two steps.

Verify the token

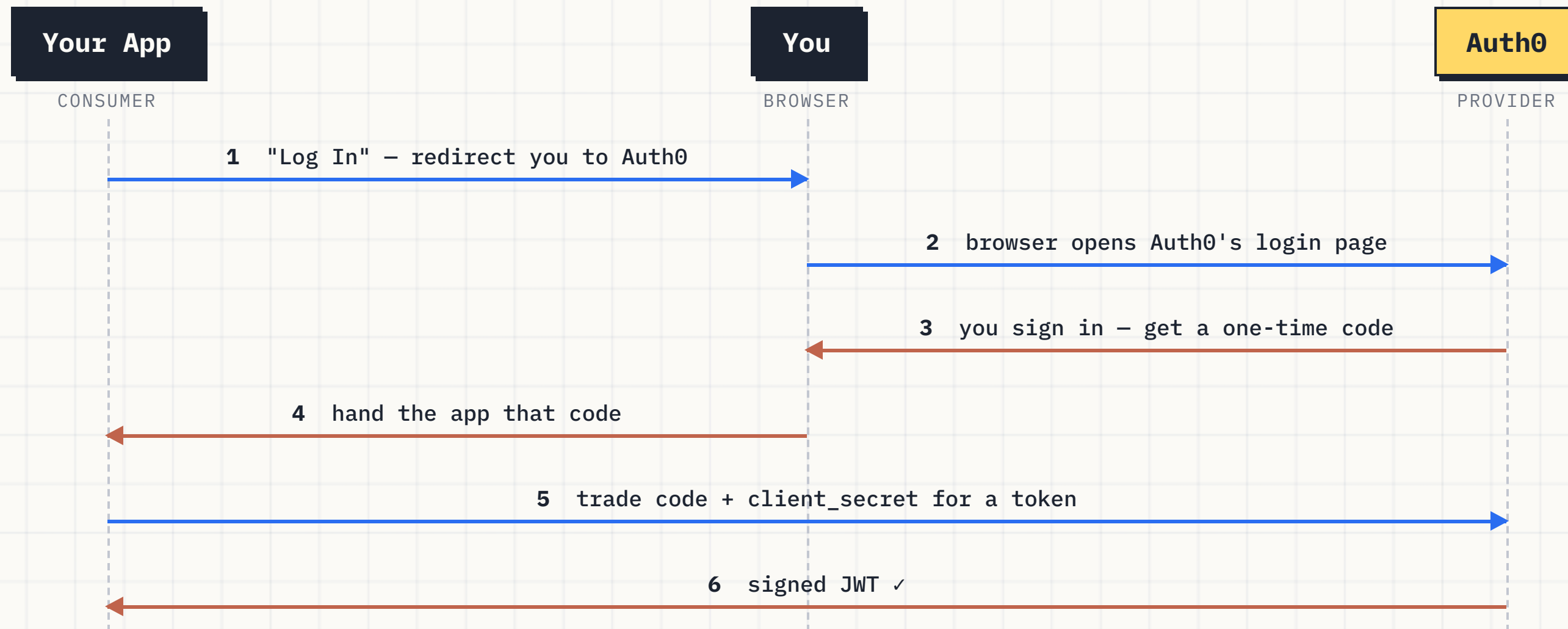


Look up the user

everything after here is Auth0 in the PERN stack.

OAUTH 2.0 – THE AUTHORIZATION-CODE FLOW

You never touch the password.



steps 3–4 happen on Auth0's own domain – your app only ever sees the code, then the token.

SETUP

Wrap the app once in a provider.

```
npm install @auth0/auth0-react

// main.jsx
<AuthProvider
  domain={import.meta.env.VITE_AUTH0_DOMAIN}
  clientId={import.meta.env.VITE_AUTH0_CLIENT_ID}
  authorizationParams={{ redirect_uri: window.location.origin }}
>
  <App />
</AuthProvider>
```

config comes from VITE_ env vars — same pattern as deployment.

THE HOOK

useAuth0 tells you who's logged in.

```
const { user, isAuthenticated, isLoading,
        loginWithRedirect, logout } = useAuth0()

if (isLoading) return <p>Loading...</p>

return isAuthenticated
  ? <button onClick={() => logout()}>Log Out</button>
  : <button onClick={() => loginWithRedirect()}>Log In</button>
```

one hook, everywhere: is someone logged in, and who?

GUARDING A PAGE

Bounce guests off private pages.

```
const ProtectedRoute = ({ children }) => {  
  const { isAuthenticated, isLoading } = useAuth0()  
  
  if (isLoading) return <p>Loading...</p>  
  if (!isAuthenticated) return <Navigate to="/" replace />  
  
  return children  
}
```

THE CLASSIC BUG

`isAuthenticated` is false while `Auth0` is still starting up. Skip the `isLoading` check and you bounce users who are logged in.

SENDING THE TOKEN

Attach the JWT to protected requests.

```
const { getAccessTokenSilently } = useAuth0()

const token = await getAccessTokenSilently()

await fetch("/api/quotes", {
  method: "POST",
  headers: { Authorization: `Bearer ${token}` },
  body: JSON.stringify(quote)
})
```

a protected request without this header gets rejected.

VERIFYING THE TOKEN

One middleware guards the writes.

```
// middleware/auth.js
const { auth } = require("express-oauth2-jwt-bearer")

const checkJwt = auth({
  audience: process.env.AUTH0_AUDIENCE,
  issuerBaseUrl: process.env.AUTH0_ISSUER
})

// public – anyone can read
router.get("/", getAllQuotes)

// protected – must be logged in
router.post("/", checkJwt, createQuote)
```

WHAT IT DOES

- ✓ Verifies the signature on every protected route.
- ✓ Missing / expired / forged → automatic 401.
- ✓ The handler never even runs.

AUTH0 HAS IDENTITY – YOUR DB HAS THE DATA

Save the user to your own database.

```
router.post("/users", checkJwt, async (req, res) => {  
  const auth0Id = req.auth.payload.sub  
  
  const [user] = await User.findOrCreate({  
    where: { auth0Id },  
    defaults: { email: req.body.email }  
  })  
  
  res.json(user)  
})
```

now your quotes, posts, and votes can belong to a real row.

OWNERSHIP

Logged in isn't the same as allowed.

```
router.delete("/:id", checkJwt, async (req, res) => {
  const me = await User.findOne({
    where: { auth0Id: req.auth.payload.sub }
  })
  const quote = await Quote.findPk(req.params.id)
  if (!quote) return res.sendStatus(404)

  // do you own it?
  if (quote.userId !== me.id) return res.sendStatus(403)

  await quote.destroy()
  res.sendStatus(204)
})
```

KNOW THE CODES

- ✓ 401 — you're not logged in.
- ✓ 403 — logged in, but not allowed.

COMMON PITFALLS

Every one of these is a real breach.

ON THE FRONTEND

- ✗ Enforcing permissions only in the UI.
- ✗ Storing tokens in localStorage (XSS).
- ✗ Forgetting the isLoading guard.

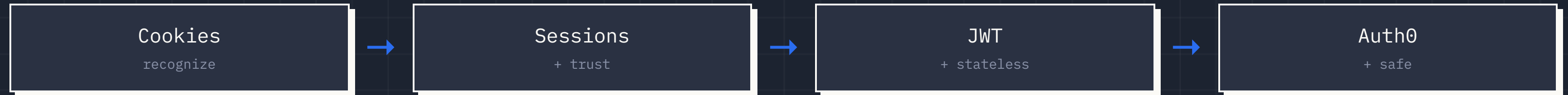
ON THE BACKEND

- ✗ One write route left unprotected.
- ✗ Checking login but never ownership.
- ✗ Trusting anything the client sends.

one unguarded write route undoes all the others.

THE WHOLE ARC

Four answers to one question.

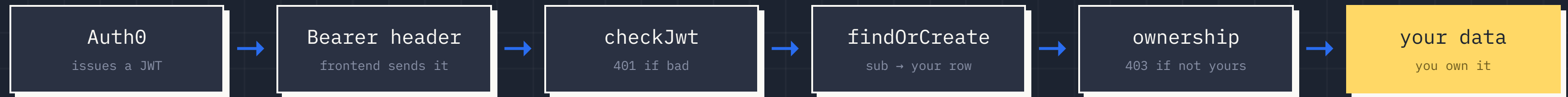


Cookies let a server recognize a browser. Sessions made it trustworthy. JWTs made it stateless. Auth0 runs the dangerous parts for us.

and credentials live in env vars — never in code.

EVERYTHING, IN ONE REQUEST

How it all fits together.



TWO QUESTIONS, EVERY ROUTE

authN — who are you? Auth0 answers. **authZ** — may you do this? Your handler answers, by checking ownership.

THE INSTINCTS TO KEEP

- ✓ **authN** ≠ **authZ** — 401 vs 403.
- ✓ Never trust the client; check on the server.
- ✓ Check ownership, not just login.
- ✓ Secrets live in env vars — never in code.

the syntax will fade — the instinct is the point: never trust the client, always check ownership.